

House Prices

Data Science

Welcome to the Data Science Project. This notebook provides you with the basic commands to start your journey. This project is going to be framed using the **CRISP-ML(Q)** (CRoss Industry Standard Process in Machine Learning with Quality Assurance).



<https://ml-ops.org/content/crisp-ml>

1. Business & Data Understanding

Developing machine learning applications starts with identifying the scope of the ML application, the success criteria, and a data quality verification. The goal of this first phase is to ensure the feasibility of the project.

Thus we invite you to read about the use case => <https://www.kaggle.com/c/house-prices-advanced-regression-techniques> Kaggle provides great tutorials about this use case. We invite you to read them.

Furthermore, we invite you to reflect on your own experience and knowledge about what makes the price of a home.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import scipy.stats as stats
from statsmodels.graphics.gofplots import qqplot

plt.style.use('seaborn-v0_8') # pretty matplotlib plots
```

```
train_raw = pd.read_csv('Data/train.csv', index_col = 'Id')
validation_raw = pd.read_csv('Data/test.csv', index_col = 'Id')
```

1.1 First look at the dataset

Let's have a look at the dataset we imported.

```
train_raw.head()
```

	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	LotShape	LandContour	Utilities	LotConfig	...	Poo
Id												
1	60	RL	65.0	8450	Pave	NaN	Reg	Lvl	AllPub	Inside	...	
2	20	RL	80.0	9600	Pave	NaN	Reg	Lvl	AllPub	FR2	...	
3	60	RL	68.0	11250	Pave	NaN	IR1	Lvl	AllPub	Inside	...	
4	70	RL	60.0	9550	Pave	NaN	IR1	Lvl	AllPub	Corner	...	
5	60	RL	84.0	14260	Pave	NaN	IR1	Lvl	AllPub	FR2	...	

5 rows × 80 columns

The dependant variable

The first thing to do is to have a look at our target variable, namely **SalePrice**. We shall need to predict it for the test dataset.

```
# descriptive statistics summary
train_raw['SalePrice'].describe()
```

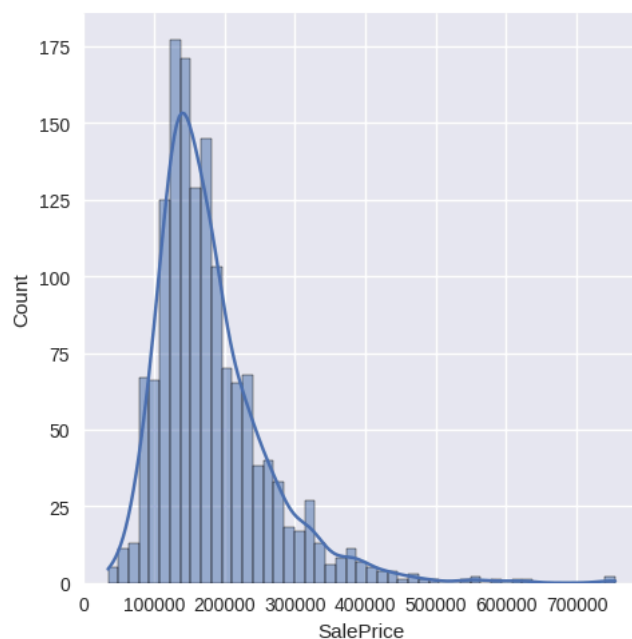
	SalePrice
count	1460.000000
mean	180921.195890
std	79442.502883
min	34900.000000
25%	129975.000000
50%	163000.000000
75%	214000.000000
max	755000.000000

dtype: float64

This isn't very visual, but we can already see that the lower quartile is around 130k, the median at 163k, the mean at 180k and the upper quartile at 214k. This means the distribution is skewed.

Let's plot the distribution to have a look.

```
# distribution plot with normal fit
ax = sns.displot(data = train_raw['SalePrice'], kde=True)
plt.show()
```



We can see the distribution is completely skewed. Let's do a quick hypothesis testing just to confirm it's not normal.

In our case, the null hypothesis states the variable **SalePrice** comes from a normally distributed population. We need to perform the **Shapiro-Wilk test** in order to reject our null hypothesis.

https://en.wikipedia.org/wiki/Shapiro%E2%80%93Wilk_test

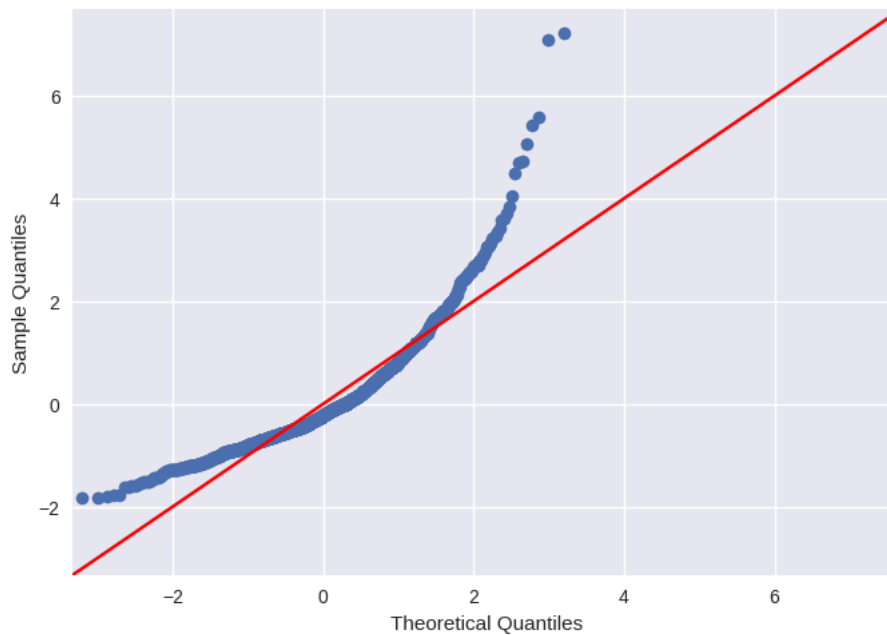
```
# Shapiro-Wilk Test for normality together with the Q-Q plot
alpha = 0.05
W, p = stats.shapiro(train_raw['SalePrice'])

print('The associated p-value is : ' + str(p))

if p < alpha :
    print('With a threshold  $\alpha = ' + str(alpha) + ', we reject the null hypothesis')
else :
    print('With a threshold  $\alpha = ' + str(alpha) + ', we fail to reject the null hypothesis')

qqplot(train_raw['SalePrice'], dist = stats.distributions.norm, fit = True, line = '45')
plt.show()$$ 
```

The associated p-value is : 3.2061412312021656e-33
 With a threshold $\alpha = 0.05$, we reject the null hypothesis



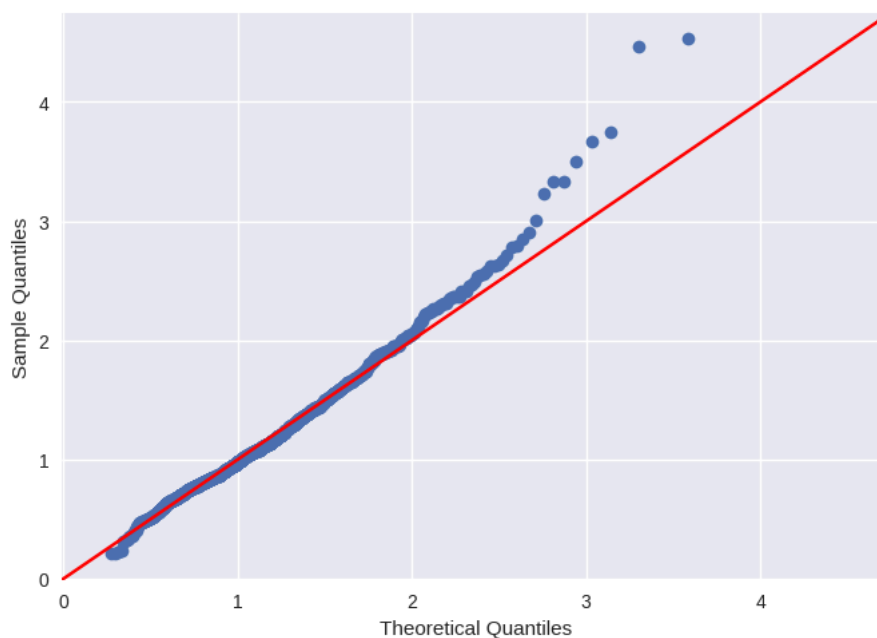
The **Shapiro-Wilk test** together with the **Q-Q Plot** confirms the variable does not come from a normally distributed population.

https://en.wikipedia.org/wiki/Q%E2%80%93Q_plot

On Kaggle, it's said that "Submissions are evaluated on Root-Mean-Squared-Error (RMSE) between the **logarithm** of the predicted value and the **logarithm** of the observed sales price. (Taking logs means that errors in predicting expensive houses and cheap houses will affect the result equally.)"

Hence let's see how does a log normal fit look like!

```
qqplot(train_raw['SalePrice'], dist = stats.distributions.lognorm, fit = True, line = '45')
plt.show()
```



It's still not perfect. Do you see these extreme values deviating on the upper right? However, it's already way better. We will probably need to remove these extreme values since regression algorithms don't behave well with these.

Let's transform our target variable SalePrice using the mapping

$$x \mapsto \log(1 + x)$$

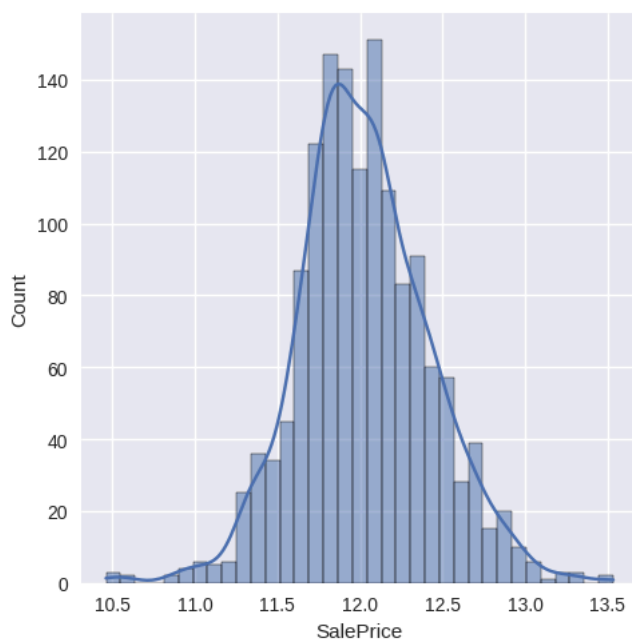
This is exactly the NumPy function **log1p**. We write the **logarithm of SalePrice** in a dedicated variable **y**.

<https://numpy.org/doc/stable/reference/generated/numpy.log1p.html>

```
# new variable target which is the logarithm of the SalePrice
```

```
target = np.log1p(train_raw['SalePrice'])

# distribution plot
ax = sns.displot(data = target, kde=True)
plt.show()
```



✓ The independant variables

Now it's time to analyse the independant variables and their interaction with the **SalePrice**. We will probably need to do some basic data cleaning first.

($\bar{o}$) . z Z

If you read carefully the description of the variables, you will see that there are three kinds of variables in the dataset :

1. Qualitative.
2. Quantitative,
3. Date.

Remark. The documentation instructs us that variables **OverallQual** and **OverallCond** are qualitative. However, we shall consider them as quantitative variables since they are already represented with values on an ordinal scale.

<http://jse.amstat.org/v19n3/decock/DataDocumentation.txt>

```
qualitative = ['MSSubClass', 'MSZoning', 'Street', 'Alley', 'LotShape', 'LandContour', 'Utilities',
               'LotConfig', 'LandSlope', 'Neighborhood', 'Condition1', 'Condition2', 'BldgType',
               'HouseStyle', 'RoofStyle', 'RoofMatl', 'Exterior1st', 'Exterior2nd', 'MasVnrType',
               'ExterQual', 'ExterCond', 'Foundation', 'BsmtQual', 'BsmtCond', 'BsmtExposure',
               'BsmtFinType1', 'BsmtFinType2', 'Heating', 'HeatingQC', 'CentralAir', 'Electrical',
               'BsmtFullBath', 'BsmtHalfBath', 'FullBath', 'HalfBath', 'BedroomAbvGr', 'KitchenQu',
               'TotRmsAbvGrd', 'Functional', 'Fireplaces', 'FireplaceQu', 'GarageType', 'GarageFinish',
               'GarageCars', 'GarageQual', 'GarageCond', 'PavedDrive', 'PoolQC', 'Fence', 'MiscFeature',
               'SaleType', 'SaleCondition', 'MoSold']

quantitative = ['LotFrontage', 'LotArea', 'OverallQual', 'OverallCond', 'MasVnrArea', 'BsmtFinSF1',
                'BsmtFinSF2', 'BsmtUnfSF', 'TotalBsmtSF', '1stFlrSF', '2ndFlrSF', 'LowQualFinSF', 'GrLivArea',
                'GarageArea', 'WoodDeckSF', 'OpenPorchSF', 'EnclosedPorch', '3SsnPorch', 'ScreenPorch',
                'PoolArea', 'MiscVal']

date = ['YearBuilt', 'YearRemodAdd', 'YrSold', 'GarageYrBlt']
```

In order to clean the data entirely, let's concatenate the **train_raw** together with the **test_raw**

```
data_raw = pd.concat([train_raw, validation_raw])
```

✓ 1.2 Imputing missing values

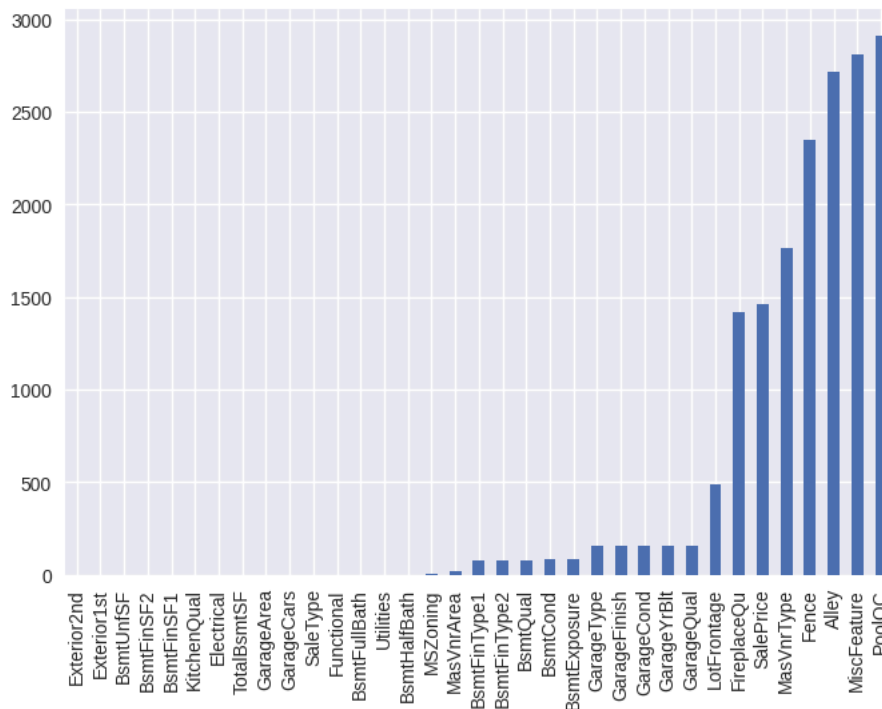
Let us first have a look at the missing variables. The following function will plot the number of NA per columns.

```
def plot_missing(data):
    missing = data.isnull().sum()
    missing = missing[missing > 0]

    if missing.empty:
        print('No missing values')
    else:
        missing.sort_values(inplace=True)
        missing.plot.bar()

    plt.show()
```

```
plot_missing(data_raw)
```



A further analysis can show that the **test.csv** contains more columns with missing values. For instance, the variable **TotalBSMSF** don't have any missing values in the training dataset, while on the holdout it does. This is typical! The training dataset is often cleaner than the production dataset.

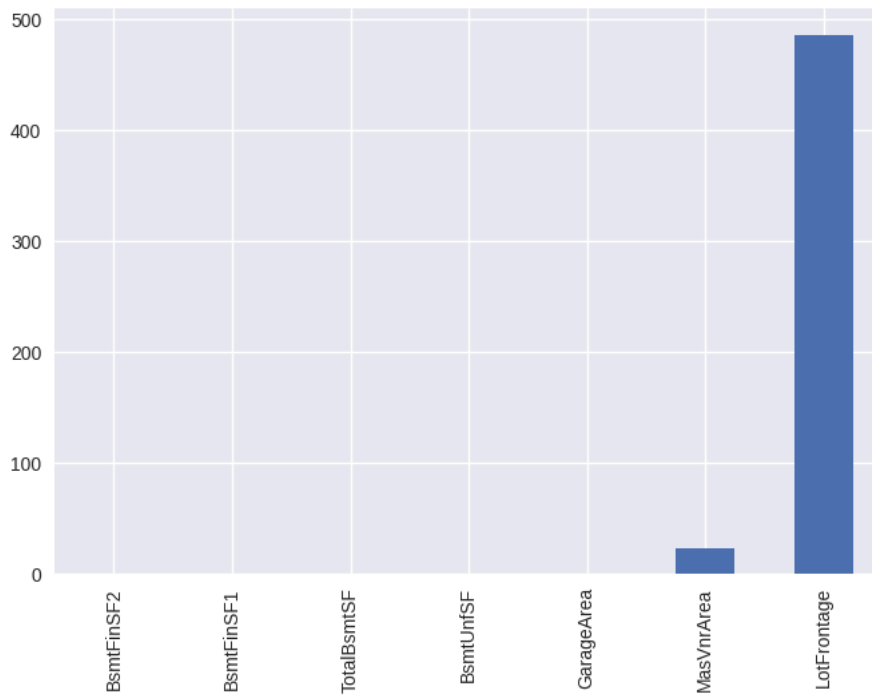
This is another reason why we put the datasets together. It's more easy to handle such phenomenon for our data analysis.

We are now going to define a strategy to "fill the blanks". It's always good to check with the business why there are missing or abnormal values. The documentation reveals that missing values are there for a good reason, because the feature does not exist.

Quantitative variables

The strategy that makes the most sens is to fill the blanks with zeros, because a missing value in the data indicates there is none.

```
plot_missing(data_raw[quantitative])
```



```
# For quantitative variables, we will fill columns with 0
def fill_missing_with_constant(data, columns, constant):

    data_clean = data.copy()

    for c in columns :

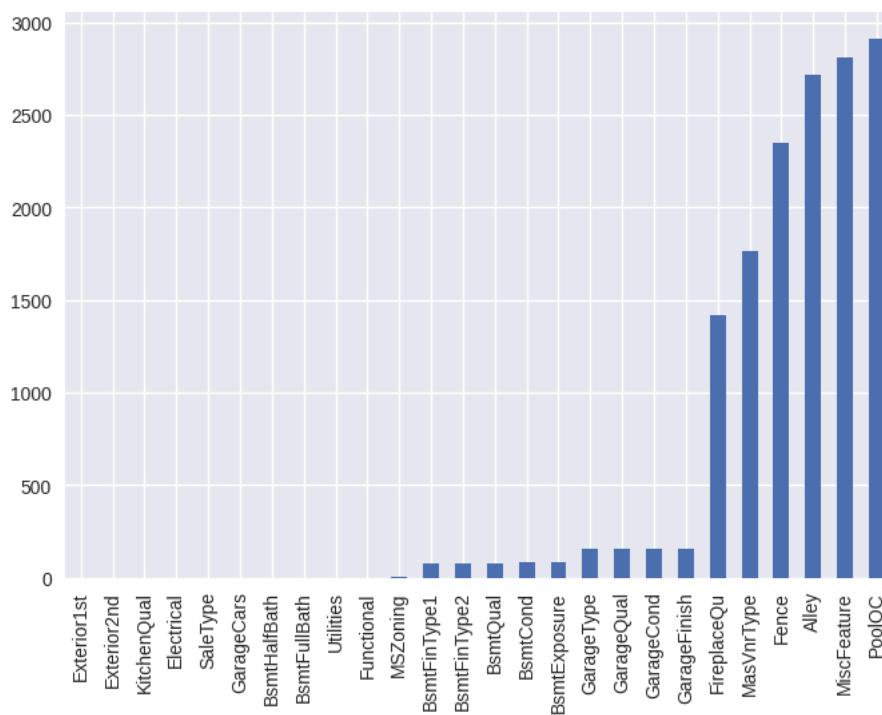
        if data_clean[c].isnull().any():
            data_clean[c] = data_clean[c].fillna(constant)

    return data_clean
```

Qualitative variables

The strategy that makes sense for qualitative variables is to fill the blank with 'NA' value like one would do in Business Intelligence.

```
plot_missing(data_raw[qualitative])
```

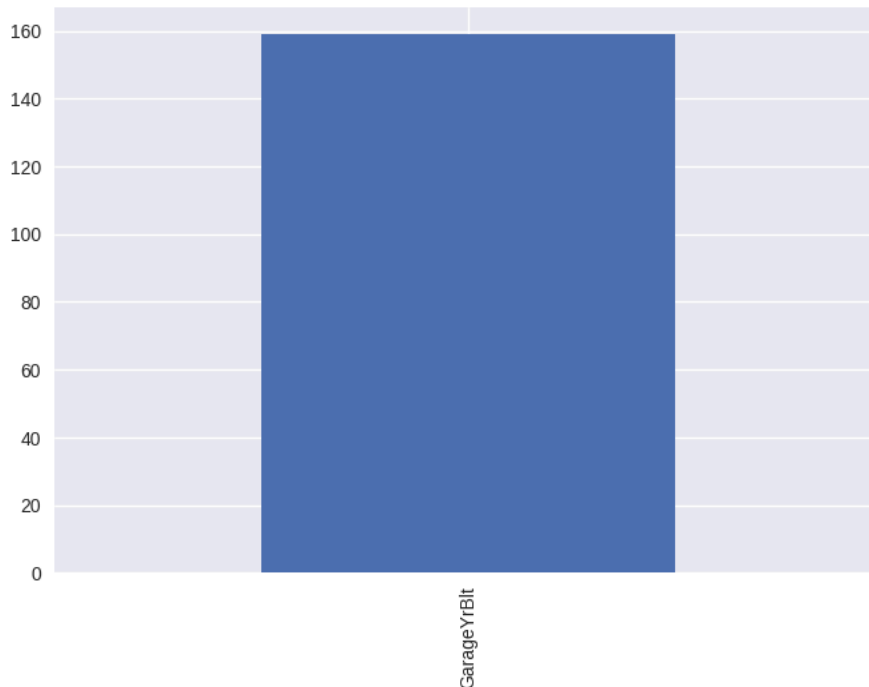


We don't need to define a particular function for that because we are using the previously defined **fill_missing_with_constant** function.

✓ Date variables

We can see that the only date variable which has missing values is the **GarageYrBlt**, probably because the house doesn't have a garage. Thus let's replace this date with the **YearBuilt** which kind of makes sense from a business perspective.

```
plot_missing(data_raw[date])
```



```
# For date variables, we will fill the missing values using the values from another column
def fill_missing_with_column(data, missing, column) :

    data_clean = data.copy()

    data_clean[missing] = np.where(data_clean[missing].isnull(), data_clean[column], data_clean[missing])

    return data_clean
```

✓ 1.3 Basic transform

Since the sales range from 2006 to 2010, it would make sense to compute the number of years since the house was built, remodeled, etc.

```
def compute_differences_to_year_sold(data) :

    data_clean = data.copy()

    data_clean['YearBuilt'] = data_clean['YrSold'] - data_clean['YearBuilt']
    data_clean['YearRemodAdd'] = data_clean['YrSold'] - data_clean['YearRemodAdd']
    data_clean['GarageYrBlt'] = data_clean['YrSold'] - data_clean['GarageYrBlt']

    return data_clean
```

✓ 1.4 Run the cleaning process

We shall now perform the cleaning of the dataset and explore the data.

```
def clean(data) :

    data_clean = data.copy()

    # imputing missing variables
```

```

data_clean = fill_missing_with_constant(data_clean, columns = quantitative, constant = 0)
data_clean = fill_missing_with_constant(data_clean, columns = qualitative, constant = 'NA')
data_clean = fill_missing_with_column(data_clean, missing = ['GarageYrBlt'], column = ['YearBuilt'])

# transform date columns
data_clean = compute_differences_to_year_sold(data_clean)

return data_clean

```

Let's run the cleaning process.

```
data_clean = clean(data_raw)
```

```
data_clean.head()
```

	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	LotShape	LandContour	Utilities	LotConfig	...	Pool
Id												
1	60	RL	65.0	8450	Pave	NA	Reg	Lvl	AllPub	Inside	...	
2	20	RL	80.0	9600	Pave	NA	Reg	Lvl	AllPub	FR2	...	
3	60	RL	68.0	11250	Pave	NA	IR1	Lvl	AllPub	Inside	...	
4	70	RL	60.0	9550	Pave	NA	IR1	Lvl	AllPub	Corner	...	
5	60	RL	84.0	14260	Pave	NA	IR1	Lvl	AllPub	FR2	...	

5 rows × 80 columns

1.5 Exploratory Data Understanding

We shall now perform the EDA and ideate on how we are going to solve this case. We have two types of variables, the categorical variables and the numeric variables. Furthermore, we need to split our cleaned dataset back into a training dataset and a validation dataset as before.

```

categorical = ['MSSubClass', 'MSZoning', 'Street', 'Alley', 'LotShape', 'LandContour', 'Utilities',
               'LotConfig', 'LandSlope', 'Neighborhood', 'Condition1', 'Condition2', 'BldgType',
               'HouseStyle', 'RoofStyle', 'RoofMatl', 'Exterior1st', 'Exterior2nd', 'MasVnrType',
               'ExterQual', 'ExterCond', 'Foundation', 'BsmtQual', 'BsmtCond', 'BsmtExposure',
               'BsmtFinType1', 'BsmtFinType2', 'Heating', 'HeatingQC', 'CentralAir', 'Electrical',
               'BsmtFullBath', 'BsmtHalfBath', 'FullBath', 'HalfBath', 'BedroomAbvGr', 'KitchenAbvGr',
               'KitchenQual', 'TotRmsAbvGrd', 'Functional', 'Fireplaces', 'FireplaceQu', 'GarageType',
               'GarageFinish', 'GarageCars', 'GarageQual', 'GarageCond', 'PavedDrive', 'PoolQC', 'Fence',
               'MiscFeature', 'SaleType', 'SaleCondition', 'YrSold', 'MoSold']

numeric = ['LotFrontage', 'LotArea', 'OverallQual', 'OverallCond', 'MasVnrArea', 'BsmtFinSF1',
           'BsmtFinSF2', 'BsmtUnfSF', 'TotalBsmtSF', '1stFlrSF', '2ndFlrSF', 'LowQualFinSF', 'GrLivArea',
           'GarageArea', 'WoodDeckSF', 'OpenPorchSF', 'EnclosedPorch', '3SsnPorch', 'ScreenPorch',
           'PoolArea', 'MiscVal', 'YearBuilt', 'YearRemodAdd', 'GarageYrBlt']

# Split of the clean dataset into train & validation
train_clean = data_clean[data_clean.index.isin(train_raw.index)]
validation_clean = data_clean[data_clean.index.isin(validation_raw.index)]

```

Numeric variables analysis

Let's first have a look at the numeric one. It's quite standard to look at the correlation between each numeric variable together with our target variable, i.e. the **logarithm of SalePrice**. Beware to not perform a correlation with the **SalePrice** itself, because the link is probably not linear as we've seen previously.

```

def correlation(y, X, features, method = 'pearson'):

    cor = pd.DataFrame()
    cor['feature'] = features

    cor['correlation_coef'] = [X[f].corr(y, method = method) for f in features]
    cor['correlation_coef'] = cor['correlation_coef'].fillna(0)

    cor = cor.sort_values('correlation_coef', ascending = False)

    plt.figure(figsize=(10, 0.25*len(features)))
    sns.barplot(data = cor, y = 'feature', x = 'correlation_coef', orient = 'h')

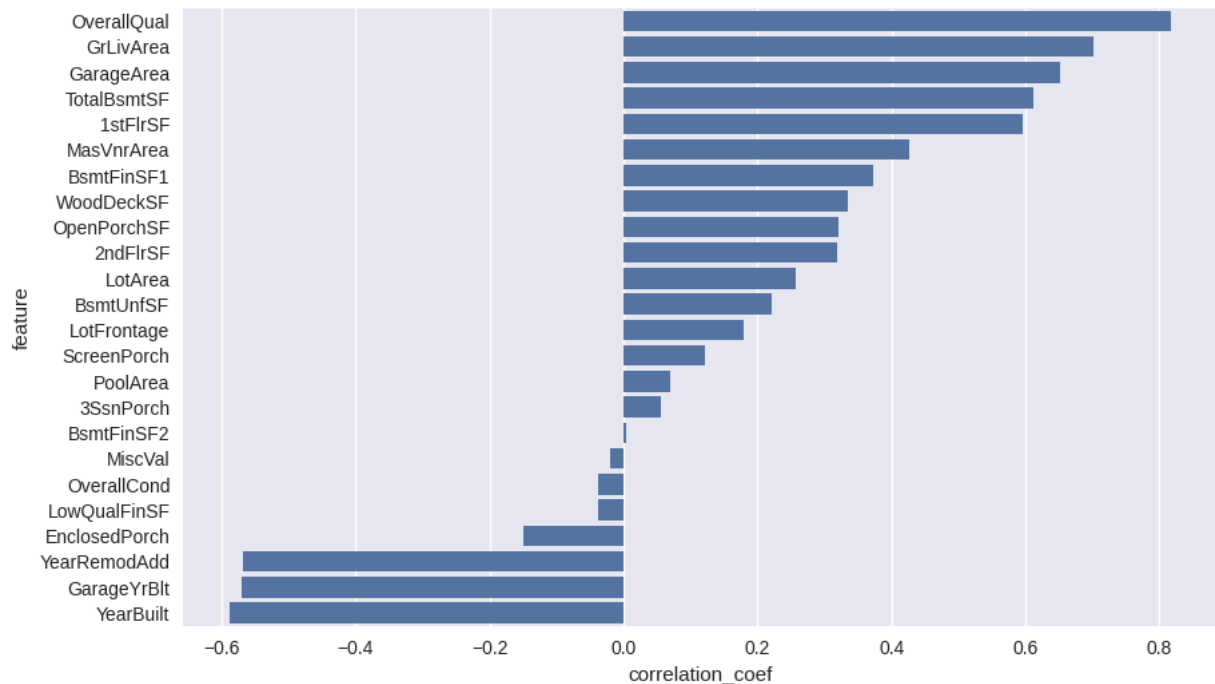
```



```
return cor
```

Let's then have a look at the correlations between the numeric variables and the **logarithm of SalePrice** over the train_clean dataset.

```
cor = correlation(target, train_clean, numeric, method = 'pearson')
```



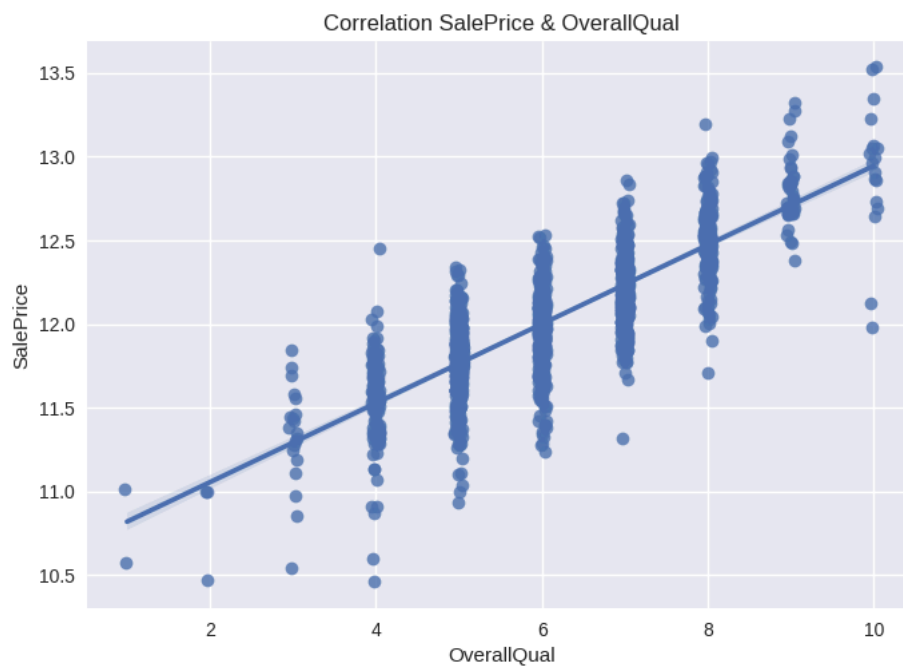
Correlation value itself can be deceiving, one must plot the scatterplots. Let's plot all the scatter plots for these numeric features.

```
def scatter_plots(y, X, features) :
    for f in features :
        x = X[f]

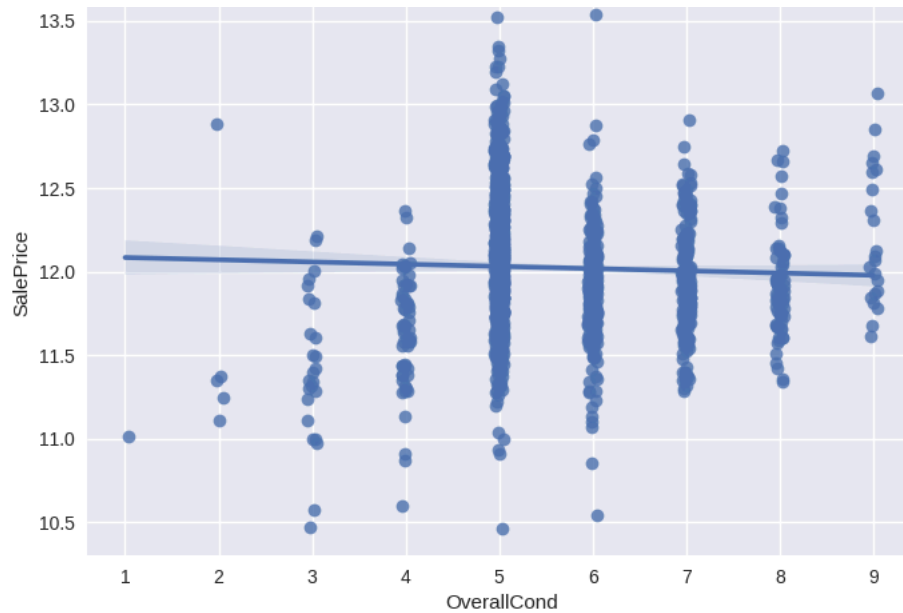
        plt.title('Correlation ' + y.name + ' & ' + x.name)
        sns.regplot(x = x.name, y = y.name, data = pd.concat([x, y], axis = 1), x_jitter = .05)

        plt.show()
```

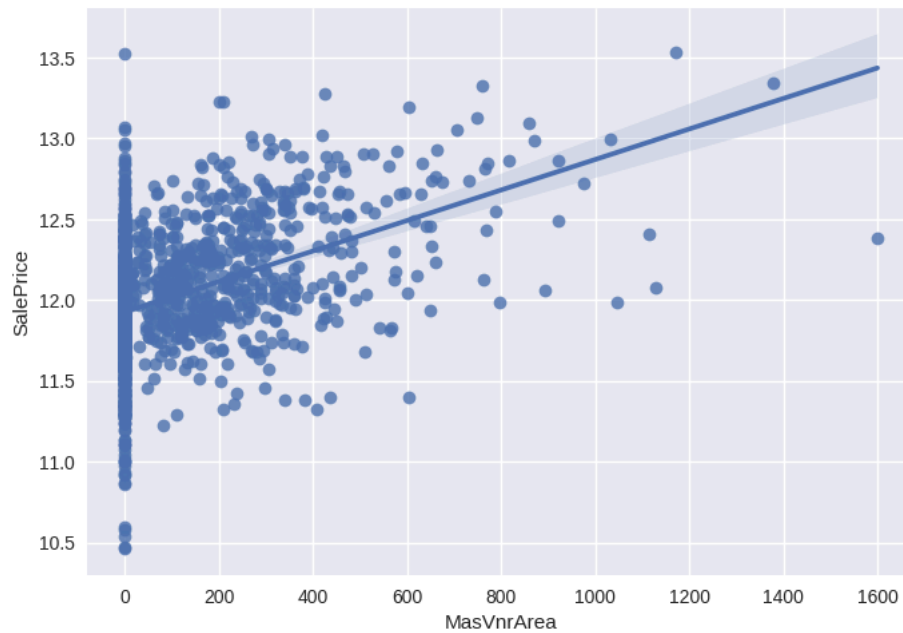
```
scatter_plots(target, train_clean, numeric)
```

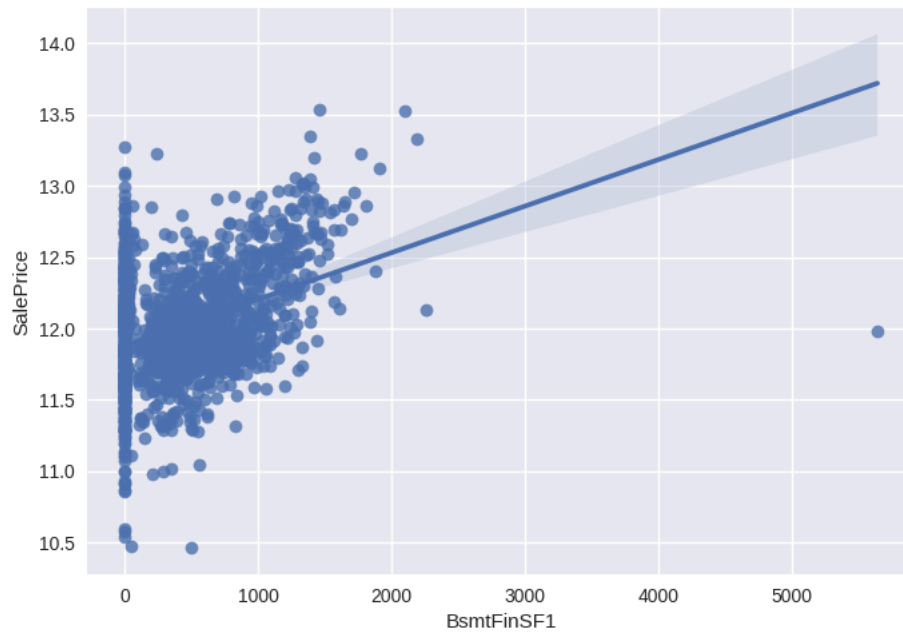
Correlation SalePrice & OverallCond



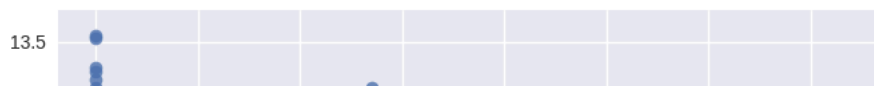
Correlation SalePrice & MasVnrArea

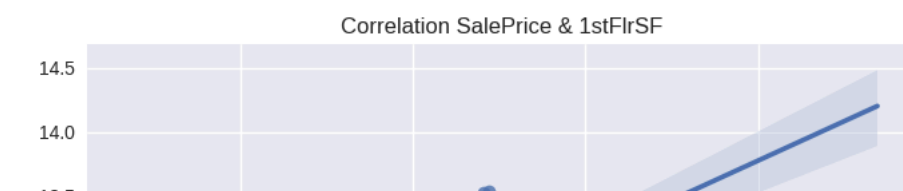
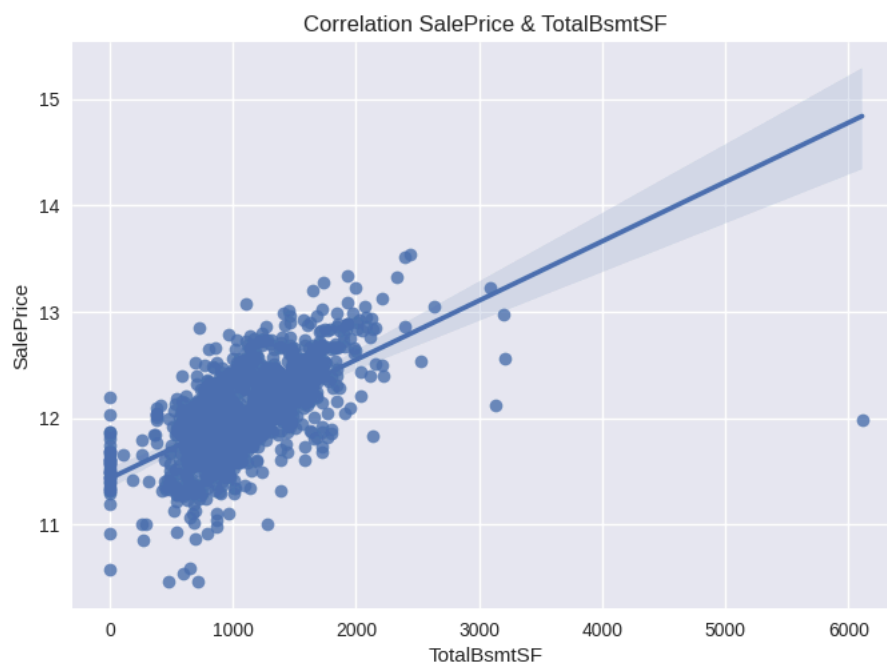
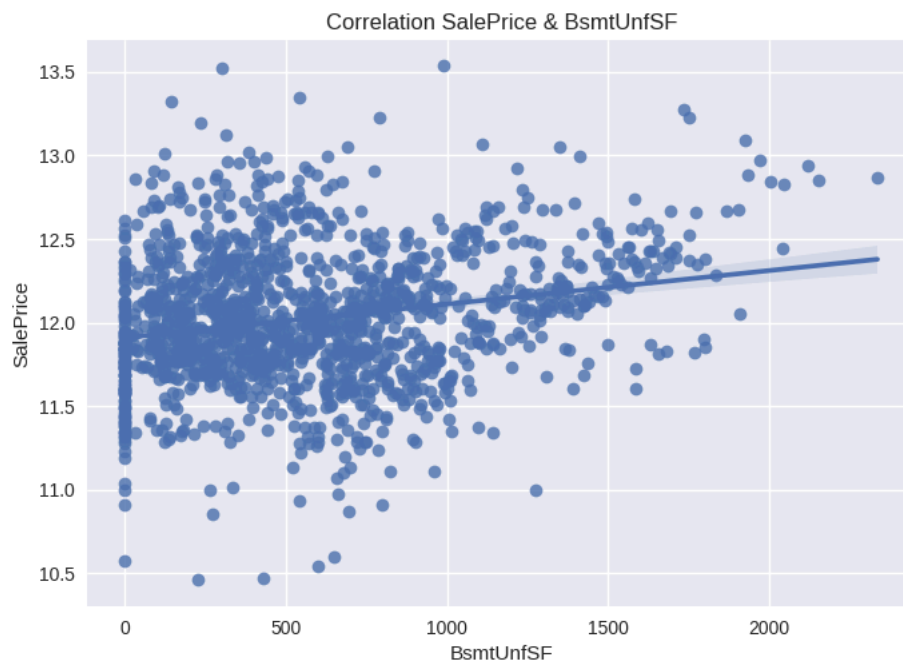
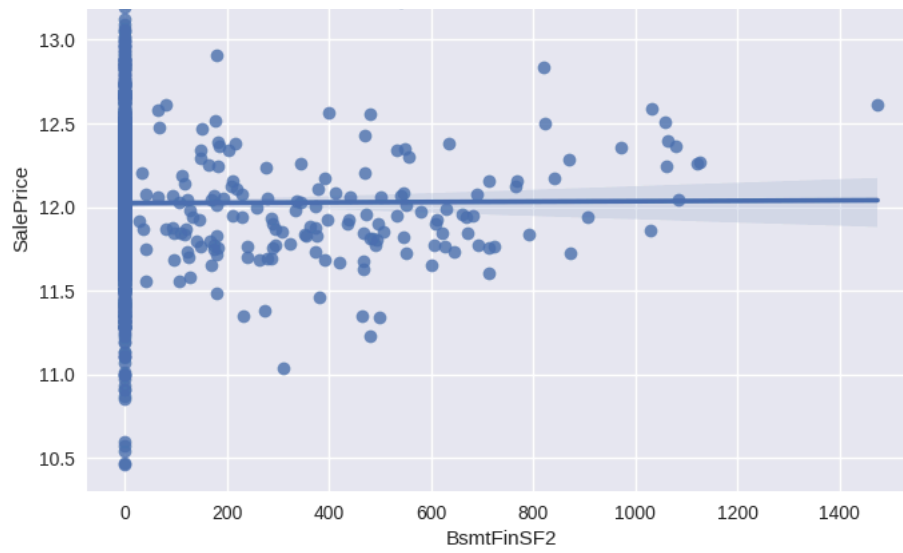


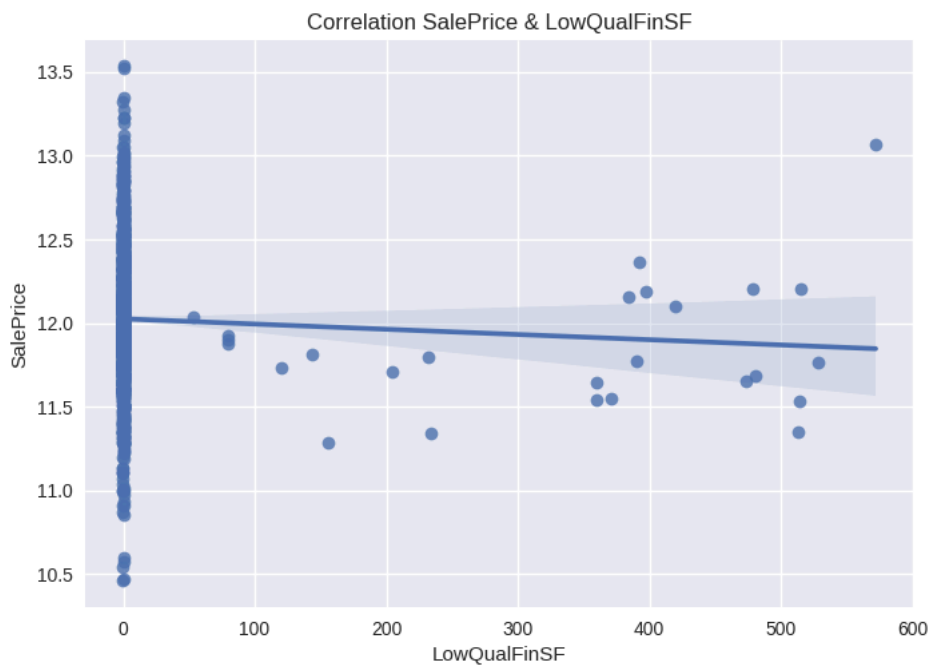
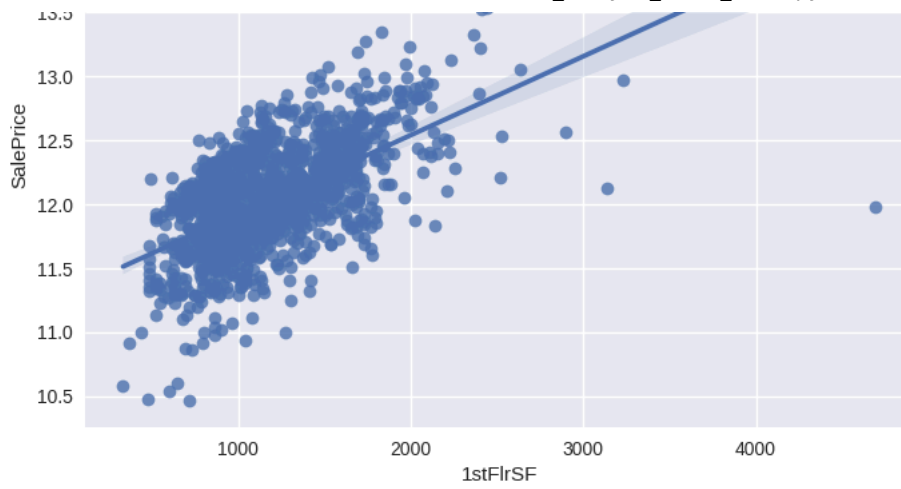
Correlation SalePrice & BsmtFinSF1

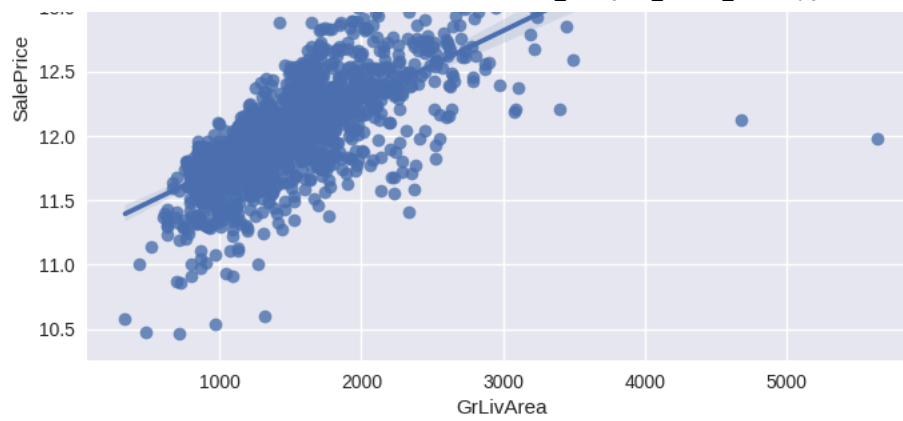


Correlation SalePrice & BsmtFinSF2

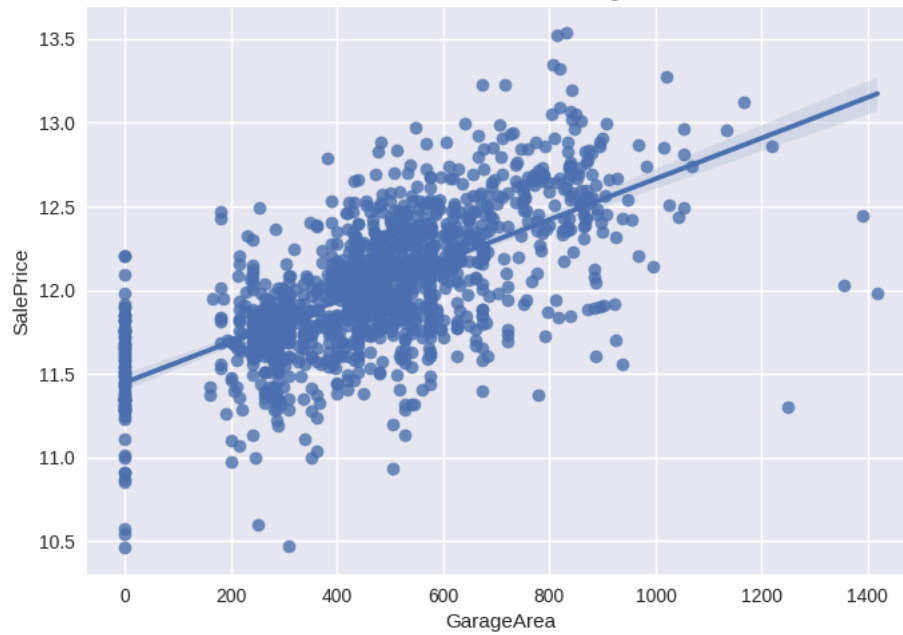




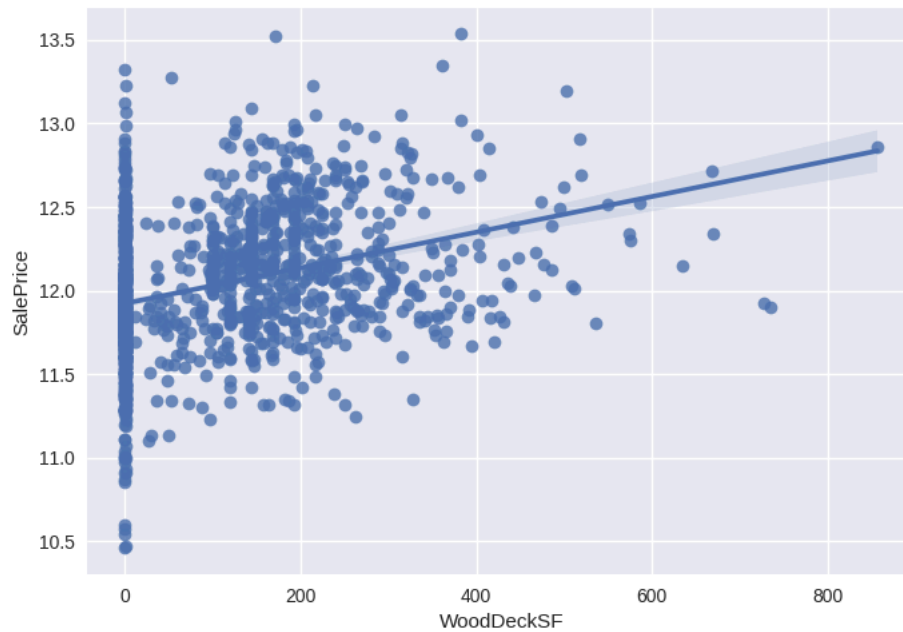




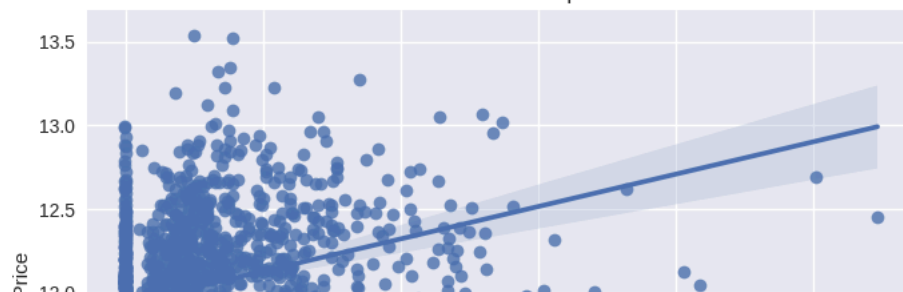
Correlation SalePrice & GarageArea

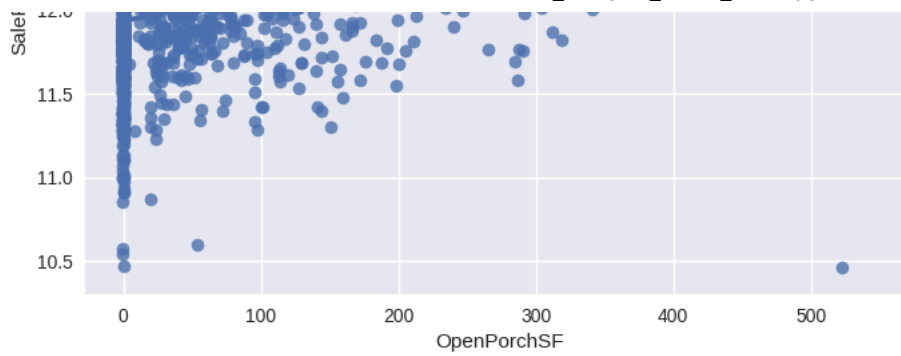


Correlation SalePrice & WoodDeckSF

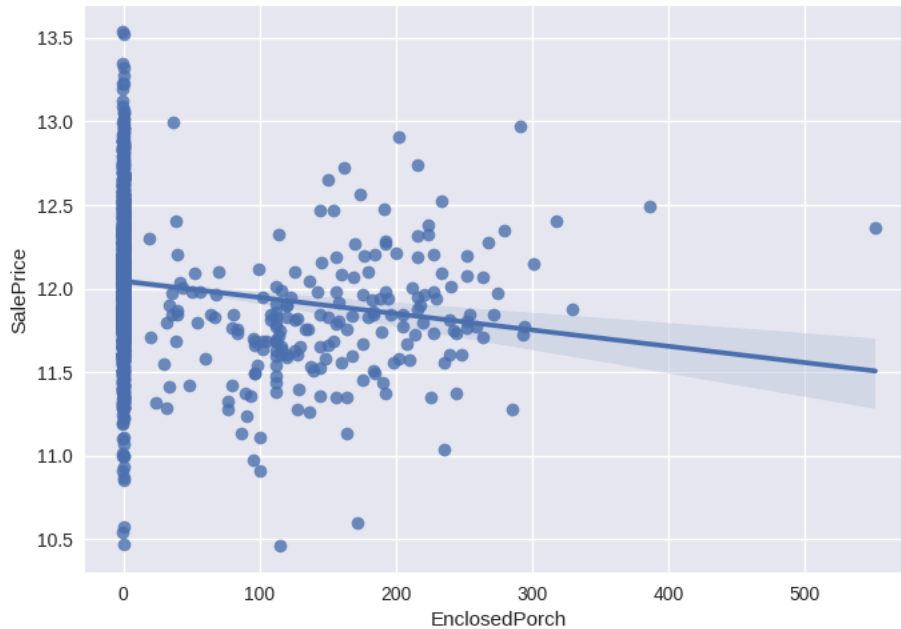


Correlation SalePrice & OpenPorchSF

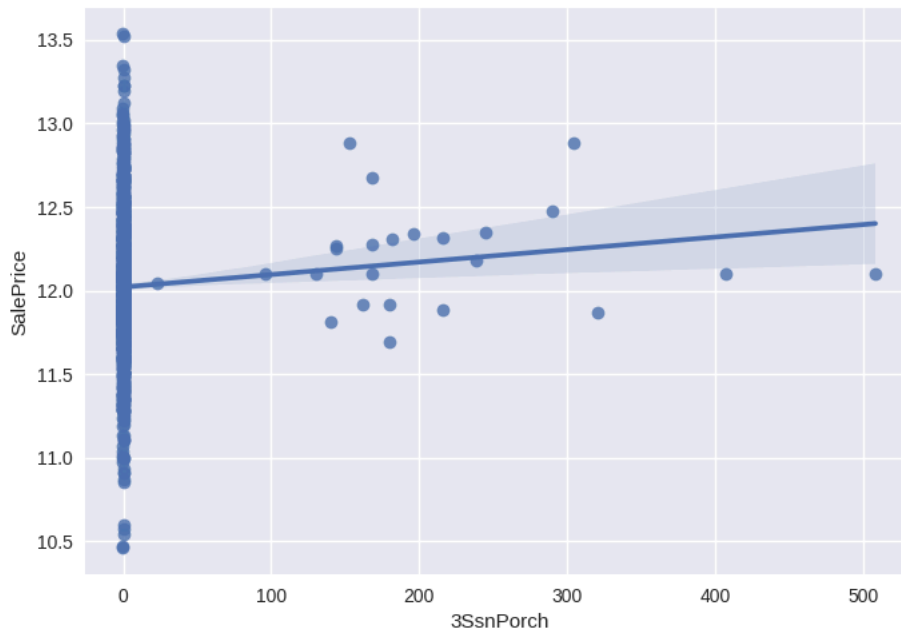




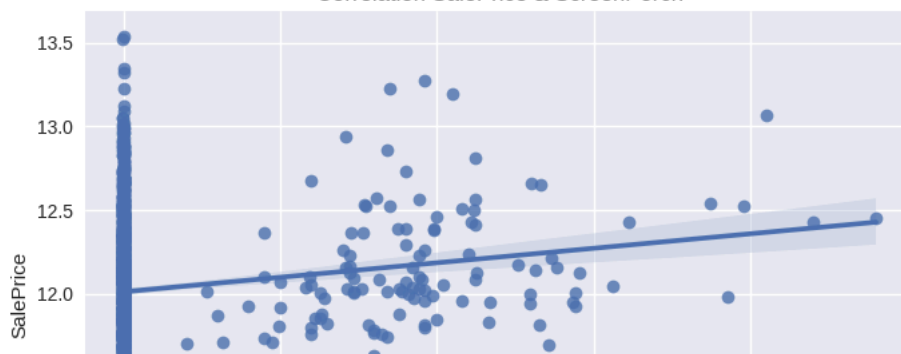
Correlation SalePrice & EnclosedPorch

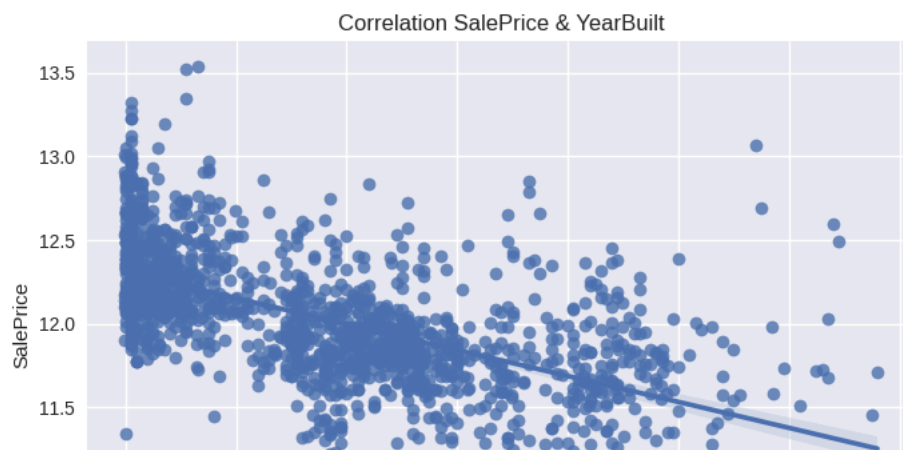
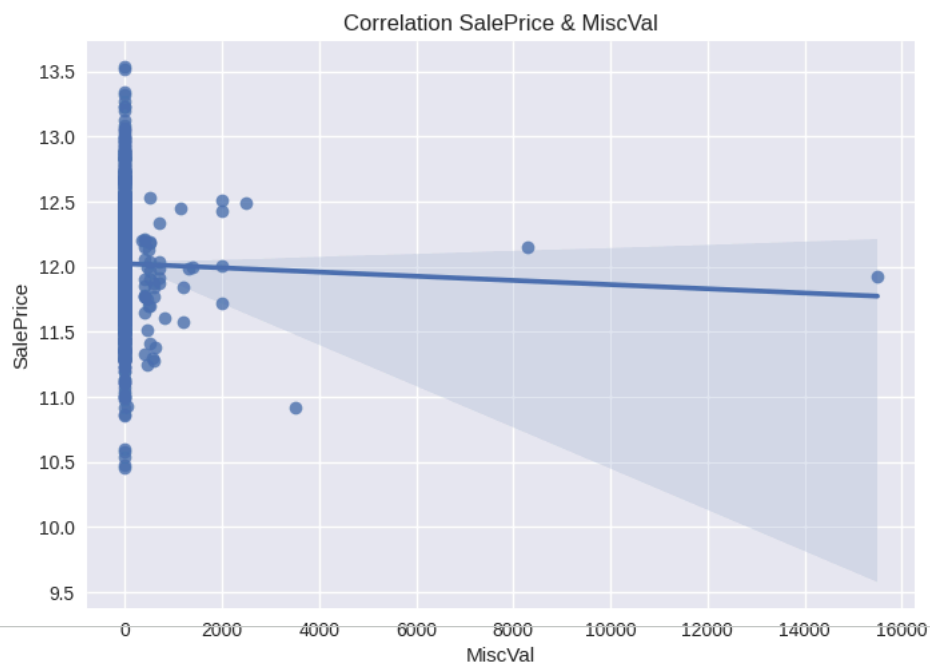
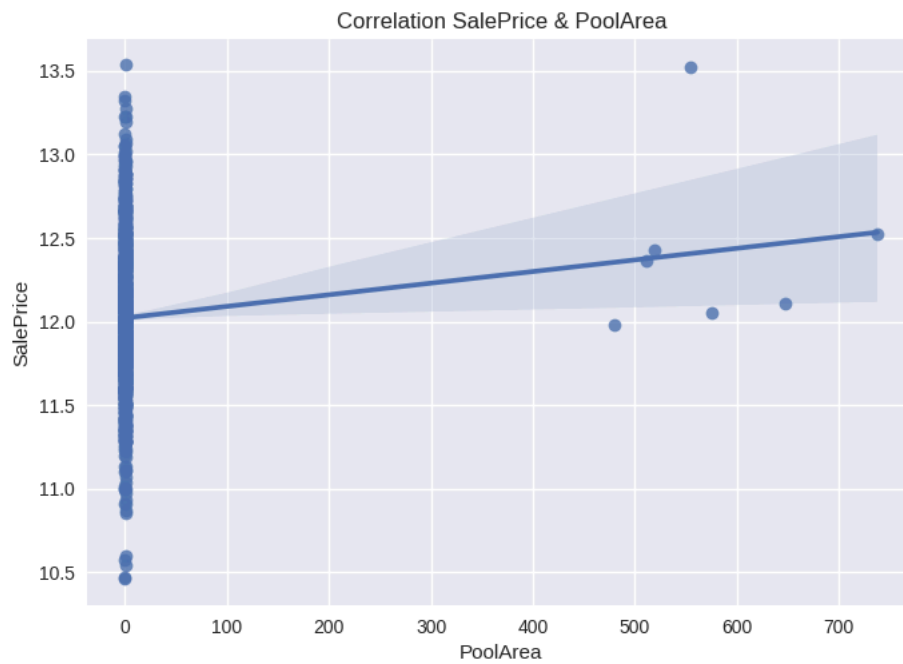
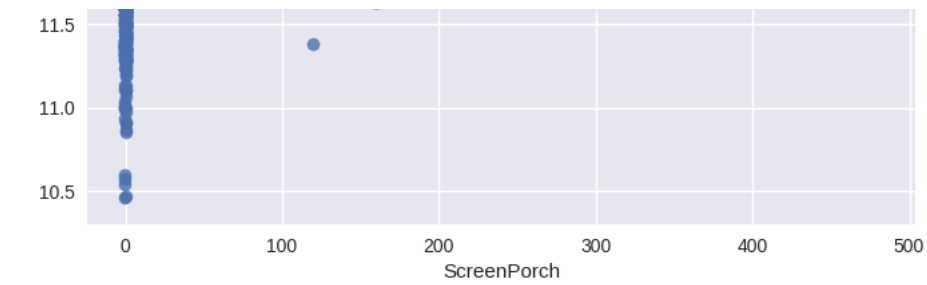


Correlation SalePrice & 3SsnPorch



Correlation SalePrice & ScreenPorch





✓ Categorical variables analysis

For categorical variables, we can't perform a correlation for obvious reasons. Thus, we need to perform an ANOVA (https://en.wikipedia.org/wiki/Analysis_of_variance) which is the counterpart in the case when we want to understand the link between a categorical variable with a numeric one.

```
def anova(y, X, features):

    anova = pd.DataFrame()
    anova['feature'] = features
    pvals = []

    for f in features:

        samples = []

        for cls in X[f].unique():

            s = y[X[f] == cls].values
            samples.append(s)

        pval = stats.f_oneway(*samples)[1]
        pvals.append(pval)

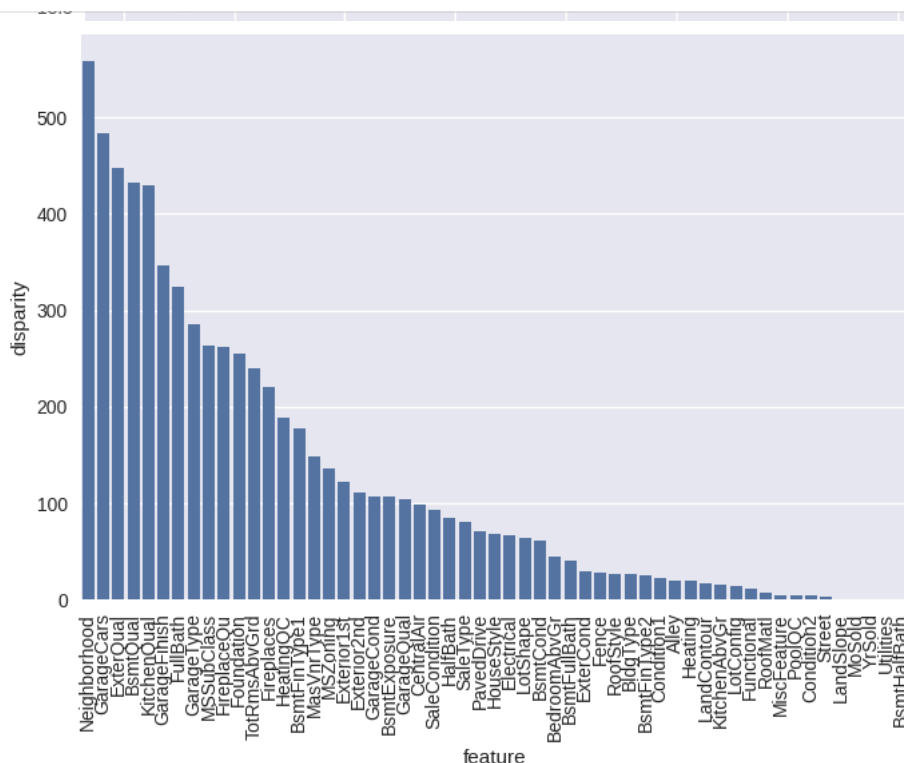
    anova['pval'] = pvals
    anova = anova.sort_values('pval')

    anova['disparity'] = np.log(1. / anova['pval'].values)

    sns.barplot(data = anova, x = 'feature', y = 'disparity')
    x = plt.xticks(rotation=90)

    return anova

anova = anova(target, train_clean, categorical)
```



```
def box_plots(y, X, features) :

    for f in features :
        x = X[f]

        plt.title('Box plot ' + y.name + ' & ' + x.name)
        sns.boxplot(x = x, y = y)
        x = plt.xticks(rotation = 90)

    plt.show()
```

```
box_plots(train_clean['SalePrice'], train_clean, categorical)
```

```
box_plot(chain_cream, categories, chain_cream, categories,
```